

Whatever data structure we use to store this description, it'll likely occupy more space than the original list itself! It would be nice if our descriptions were more lightweight.

It seems we should make `map2` lazy and have it begin immediate execution of both sides in parallel. This also addresses the problem of giving neither side priority over the other.

7.1.3 *Explicit forking*

Something still doesn't feel right about our latest choice. Is it *always* the case that we want to evaluate the two arguments to `map2` in parallel? Probably not. Consider this simple hypothetical example:

```
Par.map2(Par.unit(1), Par.unit(1))(_ + _)
```

In this case, we happen to know that the two computations we're combining will execute so quickly that there isn't much point in spawning off a separate logical thread to evaluate them. But our API doesn't give us any way of providing this sort of information. That is, our current API is very *implicit* about when computations get forked off the main thread—the programmer doesn't get to specify where this forking should occur. What if we make the forking more explicit? We can do that by inventing another function, `def fork[A](a: => Par[A]): Par[A]`, which we can take to mean that the given `Par` should be run in a separate logical thread:

```
def sum(ints: IndexedSeq[Int]): Par[Int] =
  if (ints.length <= 1)
    Par.unit(ints.headOption.getOrElse 0)
  else {
    val (l,r) = ints.splitAt(ints.length/2)
    Par.map2(Par.fork(sum(l)), Par.fork(sum(r)))(_ + _)
  }
```

With `fork`, we can now make `map2` strict, leaving it up to the programmer to wrap arguments if they wish. A function like `fork` solves the problem of instantiating our parallel computations too strictly, but more fundamentally it puts the parallelism explicitly under programmer control. We're addressing two concerns here. The first is that we need some way to indicate that the results of the two parallel tasks should be combined. Separate from this, we have the choice of whether a particular task should be performed asynchronously. By keeping these concerns separate, we avoid having any sort of global policy for parallelism attached to `map2` and other combinators we write, which would mean making tough (and ultimately arbitrary) choices about what global policy is best.

Let's now return to the question of whether `unit` should be strict or lazy. With `fork`, we can now make `unit` strict without any loss of expressiveness. A non-strict version of it, let's call it `lazyUnit`, can be implemented using `unit` and `fork`:

```
def unit[A](a: A): Par[A]
def lazyUnit[A](a: => A): Par[A] = fork(unit(a))
```

The function `lazyUnit` is a simple example of a *derived* combinator, as opposed to a *primitive* combinator like `unit`. We were able to define `lazyUnit` just in terms of other operations. Later, when we pick a representation for `Par`, `lazyUnit` won't need to know anything about this representation—its only knowledge of `Par` is through the operations `fork` and `unit` that are defined on `Par`.³

We know we want `fork` to signal that its argument gets evaluated in a separate logical thread. But we still have the question of whether it should begin doing so *immediately* upon being called, or hold on to its argument, to be evaluated in a logical thread later, when the computation is *forced* using something like `get`. In other words, should evaluation be the responsibility of `fork` or of `get`? Should evaluation be eager or lazy? When you're unsure about a meaning to assign to some function in your API, you can always continue with the design process—at some point later the trade-offs of different choices of meaning may become clear. Here we make use of a helpful trick—we'll think about what *sort of information* is required to implement `fork` and `get` with various meanings.

If `fork` begins evaluating its argument immediately in parallel, the implementation must clearly know something, either directly or indirectly, about how to create threads or submit tasks to some sort of thread pool. Moreover, this implies that the thread pool (or whatever resource we use to implement the parallelism) must be (globally) accessible and properly initialized wherever we want to call `fork`.⁴ This means we lose the ability to control the parallelism strategy used for different parts of our program. And though there's nothing inherently wrong with having a global resource for executing parallel tasks, we can imagine how it would be useful to have more fine-grained control over what implementations are used where (we might like for each subsystem of a large application to get its own thread pool with different parameters, for example). It seems much more appropriate to give `get` the responsibility of creating threads and submitting execution tasks.

Note that coming to these conclusions didn't require knowing exactly how `fork` and `get` will be implemented, or even what the representation of `Par` will be. We just reasoned informally about the sort of information required to actually spawn a parallel task, and examined the consequences of having `Par` values know about this information.

In contrast, if `fork` simply holds on to its unevaluated argument until later, it requires no access to the mechanism for implementing parallelism. It just takes an unevaluated `Par` and “marks” it for concurrent evaluation. Let's now assume this meaning for `fork`. With this model, `Par` itself doesn't need to know how to actually *implement* the parallelism. It's more a *description* of a parallel computation that gets *interpreted* at a later time by something like the `get` function. This is a shift from before, where we were considering `Par` to be a *container* of a value that we could simply

³ This sort of indifference to representation is a hint that the operations are actually more general, and can be abstracted to work for types other than just `Par`. We'll explore this topic in detail in part 3.

⁴ Much like the credit card processing system was accessible to the `buyCoffee` method in our `Cafe` example in chapter 1.